IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received January 13, 2020, accepted January 28, 2020, date of publication February 4, 2020, date of current version February 13, 2020.

Digital Object Identifier 10.1109/ACCESS.2020.2971585

A Distributed Approach to the Multi-Robot Task Allocation Problem Using the Consensus-Based Bundle Algorithm and Ant Colony System

FAROUQ ZITOUNI[1,2], SAAD HAROUS[3], (Senior Member, IEEE), AND RAMDANE MAAMRI[2,4]

[1]Department of Computer Science, Kasdi Merbah University, Ouargla 30000, Algeria
[2]LIRE Laboratory, Abdelhamid Mehri University, Constantine 25000, Algeria
[3]Department of Computer Science and Software Engineering, United Arab Emirates University, Abu Dhabi, United Arab Emirates
[4]Department of Computer Science, Abdelhamid Mehri University, Constantine 25000, Algeria

Corresponding author: Farouq Zitouni (farouq.zitouni@univ-constantine2.dz)

ABSTRACT We propose a distributed approach to solve the multi-robot task allocation problem. This problem consists of two distinct sets: robots and tasks. The objective is to assign tasks to robots while optimizing a given criterion. This problem is known to be $\mathcal{NP}$-hard even with small numbers of robots and tasks. The field of survivors' search and rescue is adopted: i.e. some Unmanned Aerial Vehicles are used to rescue a number of survivors. We choose this problem, given its importance in everyday life: (a) survivors are the tasks; (b) Unmanned Aerial Vehicles are the robots; and (c) the objective is to rescue the maximum number of survivors while minimizing the makespan (time elapsed between rescuing the first and last survivors) and traveled distances. The approach is composed of two phases: inclusion and consensus. During the inclusion phase, each Unmanned Aerial Vehicle builds a bundle of survivors using the Ant Colony System. During the consensus phase, Unmanned Aerial Vehicles resolve conflicts in their bundles of survivors (i.e. a survivor is being chosen by more than two Unmanned Aerial Vehicles), using an adequate coordination mechanism. The approach is implemented using Java programming language and JADE multi-agent Framework. The performance of our approach is compared to five state-of-the-art multi-robot task allocation solutions. Simulation results show that the proposed approach outperforms these solutions, in terms of: (i) makespans; (ii) traveled distances; and (iii) exchanged messages.

INDEX TERMS Ant colony system, consensus-based bundle algorithm, coordination, multi-robot systems, multi-robot task allocation problem, resolution of conflicts.

I. INTRODUCTION

The Multi-Robot Task Allocation (MRTA) problem is a key-concept in Multi-Robot Systems (MRS). It can be modeled as two distinct sets: a set T of tasks to be achieved and a set R of robots capable of doing these tasks. Goals of such a problem can be summarized as follows [79].

1) The set T is divided into disjoint subsets T_i.
2) The set R is divided into overlapping subsets R_j.
3) Subsets T_i are assigned to subsets R_j, while a given objective function is optimized (minimized or maximized).

The associate editor coordinating the review of this manuscript and approving it for publication was Bijoy Chand Chatterjee.

The MRTA problem can be found in several fields of MRS, such as: reconnaissance [5], [36], unmanned search and rescue missions [26], [27], logistics [23], [37], [54], autonomous exploration [12], [28], etc. The MRTA problem is known to be $\mathcal{NP}$-hard [24]. Thus, it is important to take into account that the quality of found solutions is inversely proportional to the time complexity. To motivate our research and illustrate the correlation between task allocation, multi-robot systems, and industrial applications, let us present the following scenario. We imagine an online sale company that sells an article every hour. A robot at the warehouse receives an order; finds the corresponding item; prepares and packages the item; and finally sends it to the costumer. What happens if the company sells 20 items every hour? Every minute? Every second?

In 2013, the online sale website "Amazon" sold 36.8 million items on a particularly popular shopping day. With 426 items ordered per second that day, a single robot would struggle to meet and follow all orders. If the warehouse used a team of robots (multi-robot system), then each robot should plan an efficient path, e.g. the shortest one, through the warehouse to retrieve the items to be shipped (without colliding with other robots and without taking their objects).

We propose a distributed approach to the MRTA problem, in the domain of search and rescue missions. It is based on the well-known consensus-based bundle algorithm [10], which is an established benchmark for distributed MRTA problems [9]. The approach has two main phases. During the first phase, we use Ant Colony System [19] to allow Unmanned Aerial Vehicles to select survivors to rescue. Ant Colony System is introduced to generate bundles of survivors for each UAV, given to its efficiency to handle shortest path finding problems: i.e. our main objective is to minimize makespans and traveled distances in a fully connected undirected graph. During the second phase, a coordination mechanism is used to resolve conflicts in assignments between survivors and Unmanned Aerial Vehicles. Here we resolve assignment conflicts while optimizing bundle of survivors for each UAV: i.e. the Ant Colony System is dynamically executed when survivors are removed from the system; that is to say, we periodically run the Ant Colony System on an increasingly smaller number of survivors, which guarantees maintaining the best solution: i.e. minimize makespans and traveled distances. We compared the proposed approach to five state-of-the-art multi-robot task allocation solutions [35], [36], [40], [58], [81]. Obtained results show that our approach has a better performance, in terms of makespans, traveled distances and exchanged messages. Finally, our approach will be called ACS-MRTA.

The remainder of the paper is organized as follows. In section II, we overview previous work addressing the MRTA problem. In section III, we present the proposed approach and explain its steps. In section IV, we simulate our solution, compare it to some MRTA solutions, and discuss the obtained results. In section V, we conclude our work and propose some perspectives.

II. RELATED WORK

Several approaches have been proposed to solve multi-robot task allocation problems [7], [20], [21], [33], [40], [43], [50], [51], [53], [57], [75], [78]. In general, they are categorized in three categories: market-based, optimization-based and behavioral approaches [31]. Sections II-A, II-B and II-C briefly overview some of these approaches.

A. MARKET-BASED APPROACHES

Market-based techniques are frequently used to address many multi-robot task allocation problems. This is due to multiple advantages [13], [61], [85]: (i) efficiency [11]; (ii) robustness [11], [14], [85]; (iii) scalability [11], [85]; (iv) online input [15], [85]; (v) and uncertainty handling [84]. Market-based techniques are inspired from the economic theory, and generally provide an effective way to coordinate robots' activities. They are based on auctions: i.e. assign tasks to robots considering both robots' bids and auction criteria (objective function) [85].

Market-based approaches use explicit communications for coordinations. A central unit announces the availability of tasks to be assigned and robots submit their bids. The central unit receives robots' bids and allocates tasks to robots considering a given objective function. Finally, the decision of the central unit is sent to robots [83].

There are several pioneering works that addressed MRTA problems, using market-based techniques, such as: M+ [3], First-Price Auctions [84], Dynamic Role Assignment [6], MURDOCH [23], Traderbots [13] and DEMiR-CF [52]. Authors of works in [25], [48], [79] proposed three different solutions to solve the MRTA problem in the field of search and rescue missions. Also, coalition formation were used to address this problem [71], [73]. Authors of [28] focused on formation of ad-hoc teams of heterogeneous robots in MRTA problems. They used several techniques: auctions, learning, clustering and genetic algorithms [26], [27].

Market-based techniques are well-suited for distributed robots and generally produce near-optimal solutions. However, they suffer from many disadvantages: (i) use of a central communication unit [30]; (ii) consume many resources [30]; (iii) scalability is only guaranteed for small to medium problems [13]; (iv) formulation of objective functions is difficult [6]; (v) and use of negotiation protocols [13].

B. OPTIMIZATION-BASED APPROACHES

Optimization-based techniques solve MRTA problems by looking for the optimal solution. Usually, the purpose is quantitatively expressed in the form of an objective function and constraints are codded in the form of equations [55]. Many optimization-based approaches have been proposed [4], [16], [17], [59], [72], [80].

Authors of works [1], [41] proposed two distinct solutions based on linear integer programming. The goal is to monitor a certain region and allocate tasks to robots. Works in [44], [45] used the traveling salesman problem to formulate the MRTA problem and the simulated annealing to solve it. The simulated annealing was combined with some heuristics to assign jobs to processors [29], [32]. Likewise, genetic algorithms were used to design a monitoring system, which is capable of tracking several targets and manage fires [27], [56]. Optimization by ant colony was used in [63], [66], [68] to solve some MRTA problems. This problem was also solved using the tabu-search [8], [32].

In [38], the simulated annealing and ant colony algorithms have been combined to solve both path planning and MRTA problems. Authors of the work [2] proposed some MRTA solutions and extensively tested them on some scenarios. Robots are highly heterogeneous and tasks are dynamic. Particle swarm optimization were used to propose a distributed algorithm to dynamically allocate tasks to real robots [46].

Authors of works [39], [69], [70] used swarm intelligence for task assignment in large-scale multi-robot systems. The works in [77], [82] treated the MRTA problem by handling spatial, temporal and energetic constraints. The work in [36] presented a good solution for the MRTA problem using spatial queuing. Finally, the works in [76], [81] proposed two different solutions to the MRTA problem, using different metaheuristics, such as: firefly algorithm, quantum genetic algorithms and Artificial Bee Colony, Ant Colony and Bat Algorithm.

Optimization-based techniques are also well-suited for distributed robots and generally produce near-optimal solutions in efficient way [81]. Besides, scalability is guaranteed even for a large number of robots [80]. However, they suffer from a major disadvantage: use of a central robot for decision making [30].

C. BEHAVIORAL APPROACHES AND SYNTHESIS

In this category, tasks to be performed are divided into behavioral groups (tasks of the same group are interrelated). Generally, these approaches are robust, fault-tolerant and operate in real-time, but found solutions are locally optimal. Among behavioral approaches, we cite Alliance [49], BLE [64], and ASyMTRe [60].

Gerkey and Matarić [24] proposed a formal taxonomy for distributed approaches. It uses three indicators: task, robot and allocation. The first one indicates whether a task can be Single-Robot (SR) or Multi-Robot (MR): i.e. it requires one or several robot(s) for its accomplishment. The second one indicates whether a robot can be Single-Task (ST) or Multi-Task (MT): i.e. it can perform one or several task(s) simultaneously. The third one indicates whether Assignments are Instantaneous (IA) or Time-extended (TA): i.e. allocations of tasks to robots are static or dynamic. In [34], [47], the taxonomy of Gerkey and Matarić [24] has been extended to include: (i) dependencies and constraints on tasks' scheduling; and (ii) precedence, synchronization and time windows constraints on tasks.

In fact, most real-life MRTA applications manipulate heterogeneous robots and tasks [31]. Thus, it is crucial to consider these characteristics in proposed solutions. Several features can be considered to handle robot heterogeneities, such as spatial positions, physical properties, and energetic constraints. Similarly, task heterogeneities can be characterized by their spatial positions, needs, and temporal constraints.

III. PROPOSED APPROACH

A. MATHEMATICAL FORMULATION AND ARCHITECTURE

We propose an approach to solve the multi-robot task allocation problem. We choose search and rescue missions performed by some Unmanned Aerial Vehicles (UAVs) [62], [65], [74]. A number of survivors are trapped in an environment, that is difficult to access. Each survivor requires three kind of supports: drugs (D), food (F) and evacuation (E). Each UAV can evacuate the strained person and provide at least one of the two other required resources. Each survivor is supposed to be assigned to a group of UAVs, that should be built in an optimal way.

We define a set $S = \{s_1, \ldots, s_n\}$ of n survivors and a set $U = \{u_1, \ldots, u_m\}$ of m UAVs. The aim is to rescue as many survivors as possible. The following assumptions summarize the configuration of our multi-robot task allocation problem (i.e. search and rescue scenario):

1) A task (i.e. survivor) corresponds to some UAVs visiting its location and offering the required supports to the concerned survivor. For an UAV, visiting the location of a survivor and offering the needed supports is seen as the accomplishment of the considered task. Thus, we target scenarios where:
   a) Each UAV can only deal with one survivor at a time.
   b) Some survivors may require multiple UAVs.
2) Locations of UAVs and survivors are shared and supposed to be a common knowledge between UAVs. Thus, we target scenarios where assignments are instantaneous.
3) The rescue order of survivors (priorities) is crucial: i.e. usually we start with survivors having critical conditions. Each UAV builds its own list of victims to be visited. Moreover, UAVs' lists of victims may overlap (i.e. are not disjoint) and their content is invariant over time.
4) UAVs perform their tasks asynchronously.
5) A task is performed when the required UAVs have already visited its location (the strained person gets the required supports).
6) Each UAV has limited quantities of drugs and/or food and its evacuation capacity is also finite.
7) Finally, we omit search and rescue time and we just consider navigation time between survivors.

According to assumptions 1, 2 and 3, we are dealing with the XD[ST-MR-IA] class of MRTA problems [34]. That is, each UAV can only deal with one survivor at a time (ST); some survivors may require multiple UAVs for their rescue (MR); assignments are instantaneous (IA); and the interest of an UAV for a given survivor depends not only on its own list of victims but also on the lists of victims of other UAVs in the system (XD). The objective is to maximize the number of rescued survivors, minimize distances that UAVs travel and minimize the rescuing time of survivors. Thus, the problem can be formulated as follows. Given a directed graph $G = (V, E)$, where V is the set of nodes and E is the set of edges. The structure of G might be illustrated by Figure 1 for an exemple of 4 survivors: i.e. $S = \{s_1, s_2, s_3, s_4\}$. That is, $V = \{b, s_1, s_2, s_3, s_4, h\}$ and $E = \{(\alpha, \beta) | \alpha \in V, \beta \in V, \alpha \neq \beta, (\alpha, \beta) \neq (b, h), (\alpha, \beta) \neq (h, b)\}$.

Nodes b and h of V represent barrack and hospital, respectively. Other nodes correspond to survivors $s_p \in S$. Initially, all UAVs are located at the barrack b. Then, each UAV $u_q \in U$

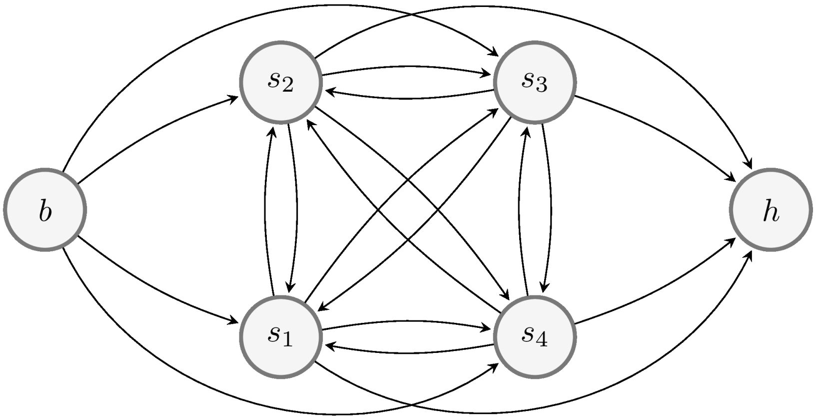

FIGURE 1. The structure of G for a set $S = \{s_1, s_2, s_3, s_4\}$ of 4 survivors.

should select some survivors to rescue and evacuate to the hospital h. It is important to take into account that each UAV must visit a node only one time. Each node $\upsilon_k \in V$ has an associated profit p_{υ_k}. If $\upsilon_k = b$ or $\upsilon_k = h$, then $p_{\upsilon_k} = 0$. The time that takes an UAV $u_q \in U$ to travel from node $\upsilon_i \in V$ to node $\upsilon_j \in V$ is denoted by $\tau^{u_q}_{\langle\upsilon_i,\upsilon_j\rangle}$, and it is computed using Equation 1.

$$\tau^{u_q}_{\langle\upsilon_i,\upsilon_j\rangle} = \frac{1}{2}\left(\frac{\Gamma^{u_q}}{a^{u_q}} + \frac{\Gamma^{u_q}}{d^{u_q}}\right) + \frac{D_{\langle\upsilon_i,\upsilon_j\rangle}}{\Gamma^{u_q}} \tag{1}$$

where
Γ^{u_q} : velocity of u_q.
a^{u_q} : acceleration of u_q.
d^{u_q} : deceleration of u_q.
$D_{\langle\upsilon_i,\upsilon_j\rangle}$: distance between nodes $(\upsilon_i, \upsilon_j) \in V^2$.

For simplicity, we suppose that the velocity of UAVs is uniform, that is to say: $\forall u_q \in U$: $a^{u_q} = \Gamma^{u_q}$ and $d^{u_q} = -\Gamma^{u_q}$. When an UAV u_q moves from node υ_i and reaches node υ_j, then a reward $\mathcal{R}(\tau^{u_q}_{\langle\upsilon_i,\upsilon_j\rangle}, p_{\upsilon_j})$ is obtained. Thus, each UAV computes a permutation σ^{u_q} of k survivors to rescue: $\sigma^{u_q} = (\sigma_1, \ldots, \sigma_k)$, with $0 < k \leq |V \setminus \{b, h\}|$. The goal is to find a solution that maximizes Equation 2.

$$\max_{\sigma^{u_q}}\{\mathcal{R}(\tau^{u_q}_{\langle b,\sigma_1\rangle}, p_{\sigma_1}) + \left[\sum_{i=1}^{k-1}\mathcal{R}(\tau^{u_q}_{\langle\sigma_i,\sigma_{i+1}\rangle}, p_{\sigma_{i+1}})\right] + \mathcal{R}(\tau^{u_q}_{\langle\sigma_k,h\rangle}, p_h)\} \tag{2}$$

Intuitively, the number of possible solutions for each UAV is: $1 + 2 + \ldots + |S|$. That is to say, it can rescue one of the $|S|$ survivors, or two of the $|S|$ survivors, and so on. Mathematically, the number of possible solutions for an UAV u_q is expressed using Equation 3.

$$\text{Number of possible solutions} = \sum_{k=1}^{n}\frac{n!}{(n-k)!} \tag{3}$$

where
n : the number of all survivors: i.e. $|S|$.
k : the number of survivors to be rescued: $|\sigma^{u_q}|$.

We denote by T^{u_q} the tour of each UAV u_q. That is: if $\sigma^{u_q} = (\sigma_1, \sigma_2, \ldots, \sigma_k)$ is the sequence of survivors to be rescued by u_q, then $T^{u_q} = \{(b, \sigma_1), (\sigma_1, \sigma_2), \ldots, (\sigma_{k-1}, \sigma_k), (\sigma_k, h)\}$. We define the variable $\mathcal{X}^{u_q}_{\langle x,y,z\rangle}$ as follows.

$$\mathcal{X}^{u_q}_{\langle x,y,z\rangle} = \begin{cases} 1, & \text{if the couple } (x, y) \text{ is used as } z^{th} \\ & \text{edge in the tour } T^{u_q}. \\ 0, & \text{otherwise.} \end{cases} \tag{4}$$

Finally, we give the mathematical formulation of our multi-robot task allocation problem using Equations 5 to 13.

$$max \sum_{(x,y)\in T^{u_q}} \sum_{z=1}^{|T^{u_q}|}\left[\mathcal{R}(\tau^{u_q}_{\langle x,y\rangle}, p_y)\mathcal{X}^{u_q}_{\langle x,y,z\rangle}\right] \tag{5}$$

Equation 5 maximizes the sum of rewards.
subject to

$$\sum_{y\in V\setminus\{b,h\}}\mathcal{X}^{u_q}_{\langle b,y,1\rangle} = 1 \tag{6}$$

Equation 6 means that the first station in the tour is the barrack and there is no path (that contains only the barrack and the hospital) between the barrack and the hospital and vice versa.

$$\sum_{x\in V\setminus\{b,h\}}\mathcal{X}^{u_q}_{\langle x,h,|T^{u_q}|\rangle} = 1 \tag{7}$$

Equation 7 means that the last station in the tour is the hospital and there is no path (that contains only the barrack and the hospital) between the barrack and the hospital and vice versa.

$$|T^{u_q}| \geq 2 \tag{8}$$

Equation 8 means that at least one survivor must be selected.

$$|T^{u_q}| \leq |S| + 1 \tag{9}$$

Equation 9 means that at most $|S|$ survivors must be selected.

$$\sum_{(x,y)\in T^{u_q}} \sum_{z=1}^{|T^{u_q}|} \mathcal{X}_{\langle x,y,z\rangle}^{u_q} = 1 \tag{10}$$

Equation 10 means that each node can only be visited once.

$$|T^{u_q}| - 1 \leq C_E^{u_q} \tag{11}$$

Equation 11 means that the evacuation capacity of u_q cannot be exceeded. $C_E^{u_q} > 0$ is the evacuation capacity of u_q.

$$\sum_{i=2}^{|T^{u_q}|} N_F^{x_i} \leq Q_F^{u_q} \tag{12}$$

Equation 12 means that the food quantity of u_q is enough. $Q_F^{u_q} > 0$ is the food quantity that u_q has and $N_F^x > 0$ is the quantity of food offered to survivor x.

$$\sum_{i=2}^{|T^{u_q}|} N_D^{x_i} \leq Q_D^{u_q} \tag{13}$$

Equation 13 means that the drugs' quantity of u_q is enough. $Q_D^{u_q} > 0$ is the drugs' quantity that u_q has and $N_D^x > 0$ is the quantity of drugs offered to survivor x.

The Consensus-Based Bundle Algorithm (CBBA) [10] is a fully distributed multi-agent task allocation algorithm, and it uses a two-phase architecture. During phase one, called inclusion phase, each UAV uses a greedy-based strategy to form a bundle of tasks. During the second phase, called consensus phase, UAVs try to resolve different conflicts between them. It is worth pointing out that CBBA is an established benchmark for comparing performances of distributed task allocation problems [9].

The two-phase architecture of CBBA is illustrated by Algorithm 1 (it is run independently on each UAV). In this work, we adopt this architecture for the allocation of tasks. Next sections describe inclusion and consensus phases.

Algorithm 1 The Fully Distributed MRTA Algorithm Run on Each UAV

```
t ← 1;
while not stopping criteria do
    Inclusion phase (local solutions): construct a list of
    tasks;
    Consensus phase (global solutions): communicate
    and resolve conflicts;
    t ← t + 1;
end
```

B. INCLUSION PHASE

During this phase, each UAV employs a greedy-based strategy to select some survivors from the set S and evacuate them to the hospital: hence, a path from node b to node h is traced in G. The goal is to build a bundle of survivors

Algorithm 2 Choice of a Survivor

input : $\eta_{\langle\upsilon_i,\upsilon_j\rangle} = \frac{1}{D_{\langle\upsilon_i,\upsilon_j\rangle}}$, visibility of the survivor υ_j, when the survivor υ_i is rescued.
input : $\pi_{\langle\upsilon_i,\upsilon_j\rangle}$, pheromone quantity of the edge (υ_i, υ_j).
input : β, parameter that regulates the influence of visibility.
input : $\mathcal{N}$, list of remaining survivors (i.e. have not been rescued).
input : θ_0, random number drawn from the interval $[0, 1]$.
output: υ_j, the selected survivor.

```
θ ⟵ random(0, 1);
/* Exploitation of the search space */
if (θ ≤ θ0) then
    The next survivor υj is selected using Equation 14;
end
/* Exploration of the search space */
else
    The next survivor υj is selected using Equation 15;
end
```

for each UAV. For example, if we consider the graph G of Figure 1, a certain UAV u might construct the tour $T^u = \{(b, s_1), (s_1, s_3), (s_3, s_4), (s_4, h)\}$, which means that its bundle of tasks contains survivors $B = \{s_1, s_3, s_4\}$, respectively.

This phase has a low computational complexity, because of the use of a greedy-based strategy [9]. However, as each UAV chooses survivors without any interaction with other UAVs, then there are likely many conflicts between them, which reduces the efficiency. We propose a strategy to construct a bundle of survivors for each UAV, using Ant Colony System (ACS) [19]. Algorithms 2 and 3 highlight this strategy.

$$j = \underset{l\in\mathcal{N}}{\operatorname{argmax}}\{\pi_{\langle\upsilon_i,\upsilon_l\rangle} \times \eta_{\langle\upsilon_i,\upsilon_l\rangle}^{\beta}\} \tag{14}$$

Equation 14 is used to choose the edge υ_i, υ_j.

$$P(j) = \frac{\pi_{\langle\upsilon_i,\upsilon_j\rangle} \times \eta_{\langle\upsilon_i,\upsilon_j\rangle}^{\beta}}{\sum_{l\in\mathcal{N}} \pi_{\langle\upsilon_i,\upsilon_l\rangle} \times \eta_{\langle\upsilon_i,\upsilon_l\rangle}^{\beta}} \tag{15}$$

Equation 15 is used to compute the choosing probability of edge υ_i, υ_j.

$$\pi_{\langle\upsilon_i,\upsilon_j\rangle} = (1-\xi)\pi_{\langle\upsilon_i,\upsilon_j\rangle} + \xi\frac{1}{|V|\varepsilon_0} \tag{16}$$

Equation 16 is used to update the pheromone quantities of the tours made by ants.

$$\pi_{\langle\upsilon_i,\upsilon_j\rangle} = (1-\delta)\pi_{\langle\upsilon_i,\upsilon_j\rangle} + \delta\frac{1}{L} \tag{17}$$

Equation 17 is used to update the pheromone quantities of the best tour.

Algorithm 2 is used by each UAV $u_q \in U$ to select the next node in the graph $G = (V, E)$. Its instructions are explained as follows.

Algorithm 3 Inclusion Phase, Run on Each UAV u_q

input : $G = (V, E)$, the graph built for each UAV u_q.
input : ε_0, cost estimation of the best solution.
input : A, set of ants.
output: The best tour $T_{+}^{u_q}$.
/* Pheromone initialization */
1 **foreach** $(\alpha \in V)$ **do**
2   **foreach** $(\beta \in V)$ **do**
3     $\pi_{\langle \alpha, \beta \rangle} \longleftarrow \frac{1}{|V|\varepsilon_0}$;
4   **end**
5 **end**
/* t_{max} is the number of iterations, e.g. $t_{max} = 1000$ */
6 **for** $(t \longleftarrow 1$ ***up to*** $t_{max})$ **do**
  /* Initially, all the ants are on the node b */
7   **foreach** $(a \in A)$ **do**
    // B contains visited nodes.
8     $B \longleftarrow \{b\}$;
9     $T^{u_q}(a, t) \longleftarrow \emptyset$;
10   **while** *(u_q is able to rescue a survivor)* **do**
11   Compute σ, the next survivor to rescue using algorithm 2;
12       $B \longleftarrow B \cup \{\sigma\}$;
13   Update the tour: $T^{u_q}(a, t) \longleftarrow T^{u_q}(a, t) \cup \{(B(|B| - 1), B(|B|))\}$;
14   Update $\pi_{\langle B(|B|-1), B(|B|) \rangle}$ using Equation 16;
15   **end**
16   Update the tour: $T^{u_q}(a, t) \longleftarrow T^{u_q}(a, t) \cup \{(B(|B|), h)\}$;
17   Update $\pi_{\langle B(|B|), h \rangle}$ using Equation 16;
18   Compute the cost $L^{u_q}(a, t)$ of the tour $T^{u_q}(a, t)$ using Equation 2;
19   **end**
20   Let $T_{+}^{u_q}$ be the best found tour;
21   **foreach** $((\alpha, \beta) \in T_{+}^{u_q})$ **do**
22   Update $\pi_{\langle \alpha, \beta \rangle}$ using Equation 17;
23   **end**
24 **end**

- Line 1: the role of θ is to switch between exploitation and exploration of search space.
- Lines 2 to 4: the next node is the one which maximizes Equation 14.
- Lines 5 to 7: the probability of choosing a node depends on its fitness. Equation 15 represents the fitness of each node. The following steps are used to select the next node:
    1) Calculate the sum of all node fitnesses in $\mathcal{N}$. Let $\mathcal{S}$ be that sum.
    2) Generate a random number in the interval $[0, \mathcal{S}]$. Let $\mathcal{R}$ be that number.
    3) Go through the set $\mathcal{N}$ and sum fitnesses. When the partial sum is greater than $\mathcal{R}$, stop and return the current node.

Algorithm 3 is used by each UAV $u_q \in U$ to construct the best bundle of survivors. Its instructions are explained as follows.

- Lines 1 to 5: pheromone matrix is initialized.
- Line 8: initially, ant $a \in A$ is at node b.
- Line 9: initially, the tour is empty.
- Line 10: the condition of the while loop means that: (i) the evacuation capacity of u_q is not exceeded; (ii) the food quantity of u_q is enough; and (iii) the drugs' quantity of u_q is enough.
- Line 13: $B(i)$ denotes the element indexed by i in the set B: i.e. the list of visited nodes.

The cost L^{u_q} of a tour T^{u_q} is computed using Equation 2. The value of $\mathcal{R}(\tau^{u_q}_{\langle \upsilon_i, \upsilon_j \rangle}, p_{\upsilon_j})$ is calculated using Equations 18 and 19.

$$p_{\upsilon_j} = e^{-\Phi_{\upsilon_j}} \tag{18}$$

$$\mathcal{R}(\tau^{u_q}_{\langle \upsilon_i, \upsilon_j \rangle}, p_{\upsilon_j}) = c_1 p_{\upsilon_j} - c_2 \left(1 - e^{-\frac{\tau^{u_q}_{\langle \upsilon_i, \upsilon_j \rangle}}{\mathcal{P}_{\upsilon_j}}}\right) \tag{19}$$

The term Φ_{υ_j} returns the time elapsed between discovering and rescuing the survivor at node υ_j: i.e. the smaller value of Φ_{υ_j} is, the larger value of p_{υ_j} is. Parameter $\mathcal{P}_{\upsilon_j}$ represents the priority of the survivor at node υ_j. Parameters c_1 and c_2 are used to influence the importance of first and second terms, respectively.

Once Algorithm 3 is executed, each UAV $u_q \in U$ possesses now its best bundle of survivors: i.e. local solutions. These local solutions might not be considered as they are, since they may contain many conflicts: e.g. a survivor is to be evacuated by several UAVs at a time. Thus, we need to resolve these conflicts and find global solutions.

C. CONSENSUS PHASE

During this phase, UAVs interact and try to resolve conflicts in their bundles of survivors. We propose an algorithm to resolve conflicts between bundles of UAVs. At the end of this phase, we obtain a set of free-conflicts bundles. Each UAV is assigned to survivors of its list. We assume that a priority value is assigned to each survivor. This value is supposed to be the same for each UAV $u_q \in U$. Below, we describe how conflicts are handled using Algorithm 4.

1) ALGORITHM 4

1) Let B^{u_q} be the bundle of survivors of UAV $u_q \in U$. Therefore, $B^{u_q} \subseteq S$.
2) If $|B^{u_q}| < |S|$, then $\widehat{B}^{u_q} = B^{u_q} \cup (S \setminus B^{u_q})$, otherwise $\widehat{B}^{u_q} = B^{u_q}$. That is, we create an augmented bundle of survivors $\widehat{B}^{u_q}$ for each $u_q \in U$. It is used to synchronize interactions between UAVs.
3) Each $u_q \in U$ sorts the survivors in $\widehat{B}^{u_q}$ according to their priorities. Let $\widehat{B}^{u_q}_{+}$ be that sorted list.
4) Each $u_q \in U$ creates an empty list $\widehat{B}^{u_q}_{-}$, that contains assigned survivors.
5) While the set $\widehat{B}^{u_q}_{+}$ is not empty, do the following steps:
    a) Let s be the first element in the set $\widehat{B}^{u_q}_{+}$.

TABLE 1. Matrix of assignments.

	s_1	s_2	s_3	s_4
u_1	$\emptyset$	$\{D,E\}$	$\{D,E\}$	$\{D,E\}$
u_2	$\{F,E\}$	$\emptyset$	$\{F,E\}$	$\{F,E\}$
u_3	$\{D,F,E\}$	$\{D,F,E\}$	$\{D,F,E\}$	$\{D,F,E\}$

b) Each $u_q \in U$ interacts with the other UAVs $\acute{u}_q \in U$.
c) The UAVs $\acute{u}_q \in U$ for which $s \notin B^{\acute{u}_q}$: (i) ignore the received signal; (ii) remove the element s from the set $\widehat{B}_+^{\acute{u}_q}$; and (iii) add the element s to the set $\widehat{B}_-^{\acute{u}_q}$.
d) The UAVs $\acute{u}_q \in U$ for which $s \in B^{\acute{u}_q}$ and s is not in conflict situation: (i) remove the element s from the set $\widehat{B}_+^{\acute{u}_q}$; and (ii) add the element s to the set $\widehat{B}_-^{\acute{u}_q}$.
e) The UAVs $\acute{u}_q \in U$ for which $s \in B^{\acute{u}_q}$ and s is in conflict situation: (i) remove the element s from the set $\widehat{B}_+^{\acute{u}_q}$; (ii) add the element s to the set $\widehat{B}_-^{\acute{u}_q}$ and (iii) resolve the conflict.

6) Each UAV $u_q \in U$ returns its bundle of survivors B^{u_q}.

In next sections, we give some definitions and examples to explain a conflict situation, and show how to deal with it.

Definition 1 (A survivor supports family): Let $s_p \in S$ be a survivor. Let $U = \{u_1, u_2, \ldots\}$ be a set of UAVs. Let $\Omega^{u_1}, \Omega^{u_2}, \ldots$ be the sets of supports of UAVs $u_q \in U$. We denote $\mathcal{F}^{s_p}$ as the supports family of survivor $s_p \in S$ and it is given as follows.

$$\mathcal{F}^{s_p} = \{\Omega^{u_q} | s_p \in B^{u_q}\}$$

where
B^{u_q} : the bundle of survivors of u_q.

Example 1: Let $S = \{s_1, s_2, s_3, s_4\}$ be a set of 4 survivors and $U = \{u_1, u_2, u_3\}$ be a set of 3 UAVs. We give: $\Omega^{u_1} = \{D, E\}$, $\Omega^{u_2} = \{F, E\}$ and $\Omega^{u_3} = \{D, F, E\}$. We assume that: $B^{u_1} = \{s_2, s_3, s_4\}$, $B^{u_2} = \{s_1, s_4, s_3\}$ and $B^{u_3} = \{s_3, s_1, s_4, s_2\}$. We summarize these assignments in Table 1.

- Survivor s_1 supports family is: $\mathcal{F}^{s_1} = \{\{F, E\}, \{D, F, E\}\}$.
- Survivor s_2 supports family is: $\mathcal{F}^{s_2} = \{\{D, E\}, \{D, F, E\}\}$.
- Survivor s_3 supports family is: $\mathcal{F}^{s_3} = \{\{D, E\}, \{F, E\}, \{D, F, E\}\}$.
- Survivor s_4 supports family is: $\mathcal{F}^{s_4} = \{\{D, E\}, \{F, E\}, \{D, F, E\}\}$.

Definition 2 (Conflict situation): Let $s_p \in S$ be a survivor and $\mathcal{F}^{s_p}$ its corresponding supports family. Survivor s_p is in a conflict situation, if and only if the condition given by Equation 20 is verified.

$$\bigcap_{\mathcal{F}_i \in \mathcal{F}^{s_p}} \mathcal{F}_i \neq \emptyset \tag{20}$$

Now, we discuss how conflicts are dealt. Let $s_p \in S$ be a survivor that is in a conflict situation. Let Ω^{s_p} denotes the supports that s_p needs (drugs (D), food (F) and evacuation (E)), that is: $\Omega^{s_p} = \{D, F, E\}$. Let $\mathcal{F}^{s_p} = \{\mathcal{F}_1, \mathcal{F}_2, \ldots\}$ be the supports family of s_p. The conflict situation of survivor s_p might be resolved if and only if the condition given by Equation 21 is verified.

$$\exists\big((\widehat{\mathcal{F}}_1 \subseteq \mathcal{F}_1), (\widehat{\mathcal{F}}_2 \subseteq \mathcal{F}_2), \ldots\big) : \\ \Big[\big[\widehat{\mathcal{F}}_1 \cup \widehat{\mathcal{F}}_2 \cup \ldots = \Omega^{s_p}\big] \wedge \big[\widehat{\mathcal{F}}_1 \cap \widehat{\mathcal{F}}_2 \cap \ldots = \emptyset\big]\Big] \tag{21}$$

Equation 21 means that all the supports that s_p needs are offered and the same support that s_p needs cannot be offered at the same time by two distinct UAVs. Thus, we need to build a new supports family $\widehat{\mathcal{F}}^{s_p} = \{\widehat{\mathcal{F}}_1, \widehat{\mathcal{F}}_2, \ldots\}$ derived from $\mathcal{F}^{s_p} = \{\mathcal{F}_1, \mathcal{F}_2, \ldots\}$, for which : $(\widehat{\mathcal{F}}_1 \subseteq \mathcal{F}_1), (\widehat{\mathcal{F}}_2 \subseteq \mathcal{F}_2), \ldots$ and Equation 21 is verified.

We return back to the instruction (5-e) of Algorithm 4. Let $U^{conflict} = \{\acute{u}_1, \acute{u}_2, \ldots\}$ bet the UAVs for which $\forall \acute{u}_q \in U^{conflict} : s_p \in B^{\acute{u}_q}$, with $U^{conflict} \subseteq U$. Let $T_+^{\acute{u}_q}$ be the tour of $\acute{u}_q \in U^{conflict}$. The conflict situation of a survivor s_p is resolved using Algorithm 5.

2) ALGORITHM 5

1) Let the couple (α, β) denotes the edge of $T_+^{\acute{u}_q}$ for which $\beta = s_p$.
2) Each $\acute{u}_q \in U^{conflict}$ computes $\mathcal{R}(\tau^{\acute{u}_q}_{\langle\alpha,\beta\rangle}, p_\beta)$, using Equation 19, and shares this value with the other UAVs in $U^{conflict}$.
3) When all the values are received, each $\acute{u}_q \in U^{conflict}$ sorts them in descending order.
4) Each support in Ω^{s_p} is assigned to the UAV $\acute{u}_q \in U^{conflict}$ which has the maximum value of $\mathcal{R}(\tau^{\acute{u}_q}_{\langle\alpha,\beta\rangle}, p_\beta)$.
5) Let $U^{unassigned} \subseteq U^{conflict}$ be the set of UAVs for which no support of s_p is assigned. Given the fact that the tour $T_+^{\acute{u}}$ of each UAV $\acute{u} \in U^{unassigned}$ is altered: i.e. the survivor s_p is no longer in its bundle of survivors ($s_p \notin B^{\acute{u}}$). So, each UAV $\acute{u} \in U^{unassigned}$ constructs a new bundle of survivors $B^{\acute{u}}$ using algorithm 3 and considering the set of survivors $S \setminus \widehat{B}_-^{\acute{u}}$.

3) ILLUSTRATIVE EXAMPLE (ALGORITHM 5)

Let $S = \{s_1, s_2, s_3, s_4\}$ be a set of 4 survivors and $U = \{u_1, u_2, u_3\}$ be a set of 3 UAVs. We give: $\Omega^{u_1} = \{D, E\}$, $\Omega^{u_2} = \{F, E\}$ and $\Omega^{u_3} = \{D, F, E\}$. We assume that: $B^{u_1} = \{s_2, s_3, s_4\}$, $B^{u_2} = \{s_1, s_4, s_3\}$ and $B^{u_3} = \{s_3, s_1, s_4, s_2\}$. We suppose that the priorities of survivors s_1, s_2, s_3 and s_4 are 2, 4, 1 and 3, respectively. Thus, $\widehat{B}_+^{u_1} = \{s_3, s_1, s_4, s_2\}$, $\widehat{B}_+^{u_2} = \{s_3, s_1, s_4, s_2\}$ and $\widehat{B}_+^{u_3} = \{s_3, s_1, s_4, s_2\}$. If we suppose $s_p = s_3$, then $U^{conflict} = \{\acute{u}_1, \acute{u}_2, \acute{u}_3\}$ and therefore we have:

1) $T_+^{\acute{u}_1} = \{(b, s_2), (s_2, s_3), (s_3, s_4), (s_4, h)\}$;
2) $T_+^{\acute{u}_2} = \{(b, s_1), (s_1, s_4), (s_4, s_3), (s_3, h)\}$;
3) $T_+^{\acute{u}_3} = \{(b, s_3), (s_3, s_1), (s_1, s_4), (s_4, s_2), (s_2, h)\}$.

UAV $\acute{u}_1$ considers the couple $(\alpha, \beta) = (s_2, s_3)$, UAV $\acute{u}_2$ considers the couple $(\alpha, \beta) = (s_4, s_3)$ and UAV $\acute{u}_3$ considers the couple $(\alpha, \beta) = (b, s_3)$ (Instruction 1 of Algorithm 5). We suppose that $\mathcal{R}(\tau^{\acute{u}_1}_{\langle s_2,s_3\rangle}, p_{s_3}) = 4$, $\mathcal{R}(\tau^{\acute{u}_2}_{\langle s_4,s_3\rangle}, p_{s_3}) = 5.3$ and $\mathcal{R}(\tau^{\acute{u}_3}_{\langle b,s_3\rangle}, p_{s_3}) = 1$ (Instruction 2 of Algorithm 5). Supports D, F and E of survivor s_3 are assigned to UAVs $\acute{u}_1$, $\acute{u}_2$ and $\acute{u}_2$, respectively (Instruction 4 of Algorithm 5). $U^{unassigned} = \{\acute{u}_3\}$; thus, UAV u_3 should construct a new bundle of tasks while ignoring survivor s_3 (Instruction 5 of Algorithm 5).

D. ANALYSIS OF THE APPROACH

In this section, we analyze the quality of solutions generated by our approach. We start with time complexity of different algorithms. Then, we discuss the convergence of the approach. Finally, we examine the optimality of the found solutions.

1) TIME COMPLEXITY OF THE APPROACH

We analyze the time complexity of Algorithms 2 and 3; and Pseudo-codes 1 and 2. For clarity, each one will be elaborated separately.

a: ALGORITHM 2

This algorithm contains three elementary instructions. When it is called by Algorithm 3, only two instructions are executed at a time: lines 1 and 3 or lines 1 and 6. Therefore, the time complexity of Algorithm 2 is $\mathcal{O}(1)$.

b: ALGORITHM 3

This algorithm contains three blocs of nested loops. Simple instructions will be omitted, since they do not really affect the time complexity of the algorithm. The first bloc starts at line 1 and ends at line 5. It contains two nested for loops and one elementary instruction. Its time complexity is $\mathcal{O}(|V|^2)$, where $|V| = |S| + 2$ and $|S|$ denotes the number of survivors. The second bloc starts at line 6 and ends at line 19. It contains essentially three nested loops and several elementary instructions. Its time complexity is $\mathcal{O}(t_{max} \times |A| \times |S|)$, where $|A|$ denotes the number of ants. The third bloc starts at line 21 and ends at line 23. It contains one instruction. Its time complexity is $\mathcal{O}(|T^u_+|)$, where $|T^u_+|$ denotes the length of the best tour of UAV u. Therefore, the time complexity of Algorithm 3 is $\mathcal{O}(|V|^2 + (t_{max} \times |A| \times |S|) + |T^u_+|)$.

c: ALGORITHM 4

This pseudo-code contains essentially a sorting phase (i.e. line 3) and one bloc of nested loops (i.e. line 5). Simple instructions will be omitted, since they do not really affect the time complexity of the pseudo-code. Fortunately, the time complexity of well-known sorting algorithms ranges from $\mathcal{O}(n \log n)$ to $\mathcal{O}(n^2)$. We use Heap Sort algorithm [42], which has $\mathcal{O}(n \log n)$ as time complexity. The time complexity of the bloc of nested loops is $\mathcal{O}(|S| \times |U|)$. Therefore, the time complexity of Algorithm 4 is $\mathcal{O}((n \log n) + (|S| \times |U|))$.

d: ALGORITHM 5

This pseudo-code contains five lines. The time complexity of line 1 is $\mathcal{O}(|T^u_+|)$, because we have a searching loop. The time complexity of line 2 is $\mathcal{O}(|U|)$, because we have a diffusion loop. The time complexity of line 3 is $\mathcal{O}(n \log n)$, because we have a sorting phase. The time complexity of line 4 is $\mathcal{O}(3 \times |U|)$, because we have a bloc of searching nested loops on UAVs and supports. The time complexity of line 5 is the same as Algorithm 3. Therefore, the time complexity of Algorithm 5 is $\mathcal{O}(|T^u_+| + |U| + (n \log n) + (3 \times |U|) + (|V|^2 + (t_{max} \times |A| \times |S|) + |T^u_+|))$.

In conclusion, we observe that the worst time complexity in our approach is $\mathcal{O}(n^3)$, which is acceptable in case of search and rescue scenarios. It is worth pointing out that a CPU that executes one billion of operations peer second takes approximatively one seconde to perform $f(n)$ operations, with $n = 1000$ (f is a given algorithm) [22].

2) CONVERGENCE OF THE APPROACH

We discuss the convergence of the proposed approach: i.e. whether it always returns a solution or not. The proposed approach follows the same scheme as the Consensus-Based Bundle Algorithm [10] and contains two phases. During the first phase we use the Ant Colony System [19] to generate local solutions. Its convergence is proven in [18]. Thus, if the values of its parameters are properly chosen, then the convergence to near-optimal local solutions is certain. The convergence of the second phase is ensured by the condition of the while loop of Algorithm 4 (i.e. line 5). We use a list $\widehat{B}^u_+$ that contains unassigned survivors. After $|\widehat{B}^u_+|$ iterations of this loop, we are sure that the list $\widehat{B}^u_+$ is empty, and therefore we have global solutions.

3) OPTIMALITY OF FOUND SOLUTIONS

According to the proven property of metaheuristics [67], one can say that the solutions produced by our approach are near-optimal. If the values of the Ant Colony System parameters are carefully chosen, then qualities of found solutions are very satisfying. Exact approaches always produce optimal solutions; however, their efficiency is very quickly deteriorated as the problem size increases [81]. In search and rescue missions the time is crucial. Our aim is to rescue survivors as soon as possible. The Ant Colony System produces acceptable solutions in an efficient way.

In addition, it is worth pointing out that the solutions generated by our approach are readjusted when conflicts are detected between bundles of survivors. In other words, when an UAV releases a survivor, then an new bundle of survivors is built to always ensure a near-optimal solution.

IV. SIMULATION, COMPARATIVE STUDY AND RESULT DISCUSSION

We simulate our approach on a number of search and rescue scenarios. We assume that the search phase is done: i.e. positions of survivors to be rescued are known by all. We used

Java programming language and JADE multi-agent Framework to perform the simulation. In section IV-A, we present a brief overview of five MRTA solutions, used for the comparative study. In section IV-B, we explain the experimental setup. In section IV-C, we discuss and analyze the obtained results.

A. SOLUTIONS USED FOR COMPARISONS

We compared the performance of our approach to five state-of-the-art MRTA solutions [35], [36], [40], [58], [81]. We performed the comparison based on the following three metrics: (i) makespan; (ii) traveled distances; and (iii) exchanged messages. A brief description of each algorithm is presented next.

1) HUNGARIAN ALGORITHM (HA) [35]

This solution takes initial locations of survivors and UAVs as input, and generates a solution that minimizes the distance traveled by each UAV. It is an offline algorithm, that is to say: it works with a static view of the environment and does not consider changes occurring during allocations. The numbers of survivors and UAVs must be equal to use this algorithm. If this is not the case, then it is iteratively applied for equal sets of survivors and UAVs. At each iteration, survivors are assigned to UAVs. If survivors are less/greater than UAVs, dummy survivors/UAVs are added to satisfy the equality constraint.

2) DECENTRALIZED GREEDY ALGORITHM (DGA) [58]

Each UAV uses a Contract Net Protocol to select survivors. UAVs share only locations of survivors. At each round, UAVs broadcast bids and receive bids from other UAVs. A bid corresponds to the closest distance between an UAV and a survivor. If a given UAV has the lowest bid for a survivor, then the latter is allocated to the former. After that, a message is broadcasted to notify everyone about this assignment. This operation is repeated until all survivors are assigned.

3) REPEAT AUCTIONS ALGORITHM (RAA) [40]

Each UAV computes a value of utility for each survivor. The utility value corresponds to the difference between the inverse of Euclidean distance (separating an UAV and a survivor) and the cost to rescue the survivor (initially set to zero). A bid value corresponds to the difference between the two highest utility values. It is added to the current survivor cost. Survivor costs are always shared between UAVs. If an UAV is outbided, then the process is repeated until the maximum value of utility becomes negative. Thus, the UAV has nothing to gain and waits for the next auction. When a stable assignment is determined, the auction round is over and winning UAVs are assigned to their survivors.

4) SPATIAL QUEUING-BASED ALGORITHM (SQA) [36]

Each UAV builds a list of preferred survivors, based on its location and distances between survivors. A bid value corresponds to the product of the inverse of Euclidian distance (separating an UAV and a survivor) and the probability of selecting a survivor. Bid values are shared between UAVs. An auction-based mechanism is used to assign survivors to UAVs and resolve different conflicts: a survivor is assigned to UAVs with the highest bid.

TABLE 2. The configuration of the ant colony system [22].

Symbol	Value
β	3
ξ	0.1
δ	0.1
θ_0	0.9
$\|A\|$	10

5) FIREFLY ALGORITHM-QUANTUM ARTIFICIAL BEE COLONY-MULTI ROBOT TASK ALLOCATION (FA-QABC-MRTA) [81]

It is an auction-based solution. It considers an auctioneer, a set of bidders, and a set of goods (a particular UAV is the auctioneer, the rest of UAVs are the bidders, and survivors are the goods). The Contract Net Protocol is used for exchanging messages between the auctioneer and bidders. First, the auctioneer communicates survivors, to be rescued, to bidders and receives their bids (using the firefly algorithm); then it decides the best assignments between UAVs and survivors (using quantum genetic algorithm and artificial bee colony); finally, it notifies bidders.

B. EXPERIMENTAL SETUP

To compare the performance of chosen solutions, we consider a 2D environment, in which there are UAVs and survivors (i.e. tasks to be assigned). Locations of survivors are shared among the UAVs. Numerical values (i.e. locations of survivors and UAVs, supports, velocities, etc.) used by our simulations are randomly generated on a 2D grid of 20×20 cells. The number of UAVs ranges from 5 to 20: i.e. 5, 10, 15 or 20 UAVs. The number of survivors varies from 6 to 24: i.e. 6, 12, 18 or 24 survivors. The velocity value of each UAV is the same for all UAVs. Thus, we have 16 different simulation scenarios because of all possible combination of the number of survivors and UAVs considered ($\{5, 6\}$, $\{5, 12\}$, $\{5, 18\}$, $\{5, 24\}$, $\{10, 6\}$, $\{10, 12\}$, $\{10, 18\}$, $\{10, 24\}$, $\{15, 6\}$, $\{15, 12\}$, $\{15, 18\}$, $\{15, 24\}$, $\{20, 6\}$, $\{20, 12\}$, $\{20, 18\}$, $\{20, 24\}$). To minimize variances, each simulation was performed 20 times. To allow a systematic comparison of performance of chosen solutions, initial locations of survivors and UAVs are the same for each couple: (solution,environment). Finally, values of the Ant Colony System parameters are summarized in Table 2.

C. EXPERIMENTAL RESULTS

The makespan is the time elapsed between the rescuing of first and last survivors in the environment. In other words, it is the time taken to rescue all survivors. Figures 2, 3, 4 and 5 show a comparison of solutions "HA", "DGA", "RAA",

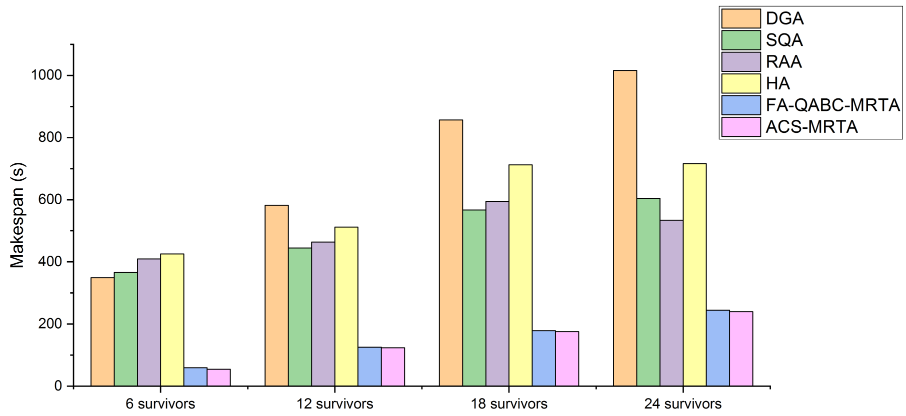

FIGURE 2. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of makespans (Case of 5 UAVs).

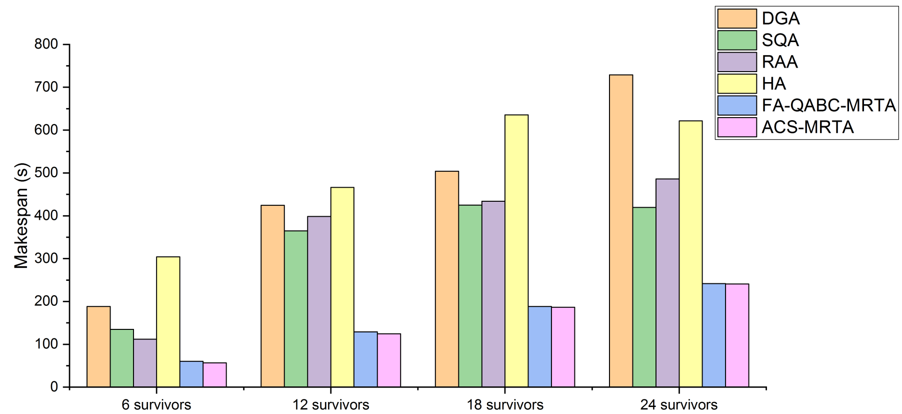

FIGURE 3. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of makespans (Case of 10 UAVs).

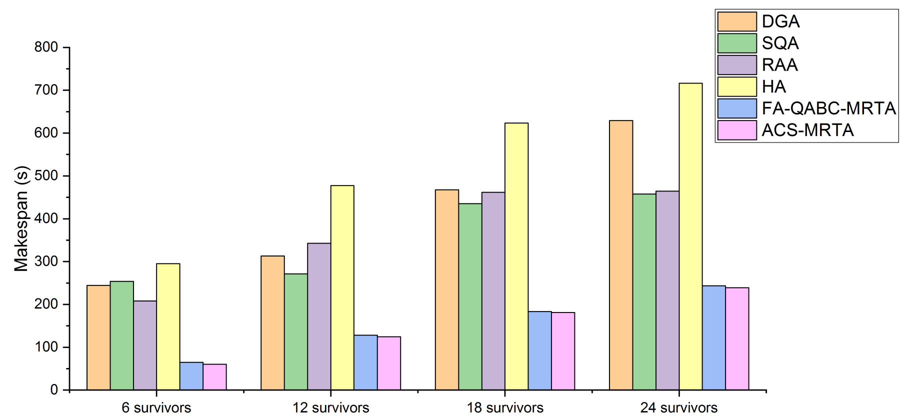

FIGURE 4. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of makespans (Case of 15 UAVs).

"SQA", "FA-QABC-MRTA" and "ACS-MRTA", in terms of makespans. The number of UAVs is set to 5, 10, 15 or 20. The number of survivors is set to 6, 12, 18 or 24. It should be noted that the makespan is proportional and inversely proportional to the numbers of survivors and UAVs, respectively: i.e. makespan increases when the number of survivors increases, and decreases when the number of UAVs increases. First, we observe that solutions "SQA" and "RAA" have almost the same makespans for all simulations. Also, we notice that makespans of solutions "DGA" and "HA" gradu-

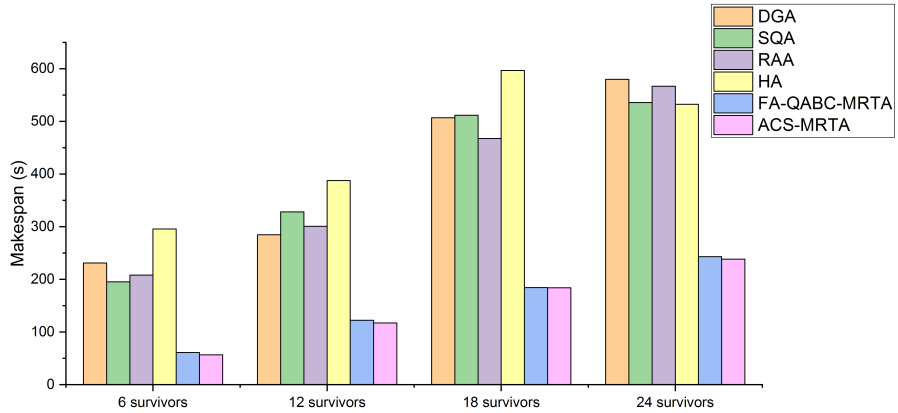

FIGURE 5. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of makespans (Case of 20 UAVs).

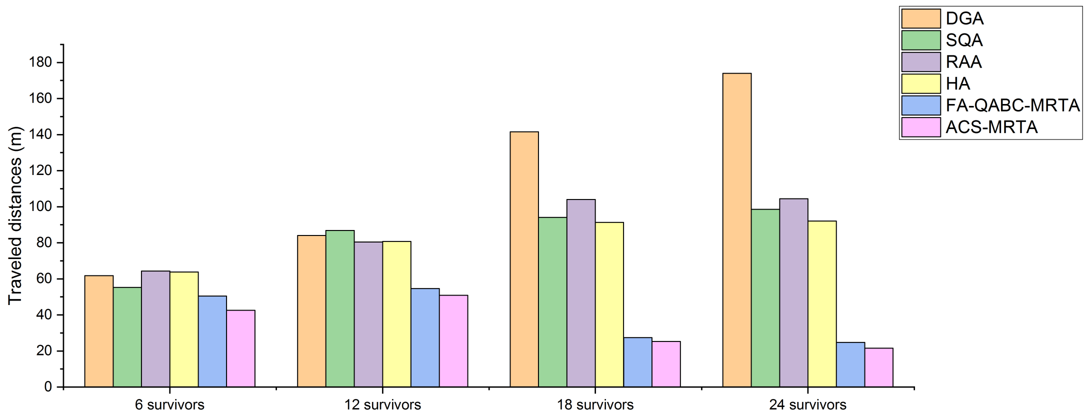

FIGURE 6. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of traveled distances (Case of 5 UAVs).

ally increase as the problem size increases. "FA-QABC-MRTA" has good makespans, but not lower than "ACS-MRTA" which has the best makespan for all simulation scenarios.

The distance traveled by UAVs is an important metric for the evaluation of MRTA solutions, that is to say: minimizing distances means extending the autonomy of UAVs. Figures 6, 7, 8 and 9 show a comparison of solutions "HA", "DGA", "RAA", "SQA", "FA-QABC-MRTA" and "ACS-MRTA", in terms of traveled distances. The number of UAVs is set to 5, 10, 15 or 20. The number of survivors is set to 6, 12, 18 or 24. It should be noted that traveled distances are proportional and inversely proportional to the number of survivors and UAVs, respectively: i.e. distances increase when the number of survivors increases, and decrease when the number of UAVs increases. First, we notice that solutions "SQA", "RAA", "HA", "FA-QABC-MRTA" and "ACS-MRTA" produce almost the same traveled distances for all simulation scenarios. We observe that traveled distances produced by the solution "DGA" progressively increase as the number of survivors increases. Finally, the solution "ACS-MRTA" generates the best traveled distances for all simulation scenarios.

It is worth pointing out that the main reason why ACS-MRTA generates the better makespans and traveled distances is the use of Ant Colony System. Actually, this algorithm is known to be an efficient method to find the best solution to any problem that can be expressed as a shortest path finding problem: i.e. minimize makespans and traveled distances. Besides, the algorithm is dynamically executed when survivors are removed from the system; that is to say, we periodically run the ACS on an increasingly smaller number of survivors, which guarantees maintaining the best solution: i.e. minimize makespans and traveled distances.

We discuss the efficiency of HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA. We consider makespans to assess the efficiency of these methods (see Figures 2, 3, 4 and 5). All methods are quite efficient: the worst makespan is achieved by DGA and it is approximately 1000 seconds. However, makespans tend to decrease as the number of UAVs increases, which is completely natural. Finally, FA-QABC-MRTA and ACS-MRTA are visibly the most efficient,

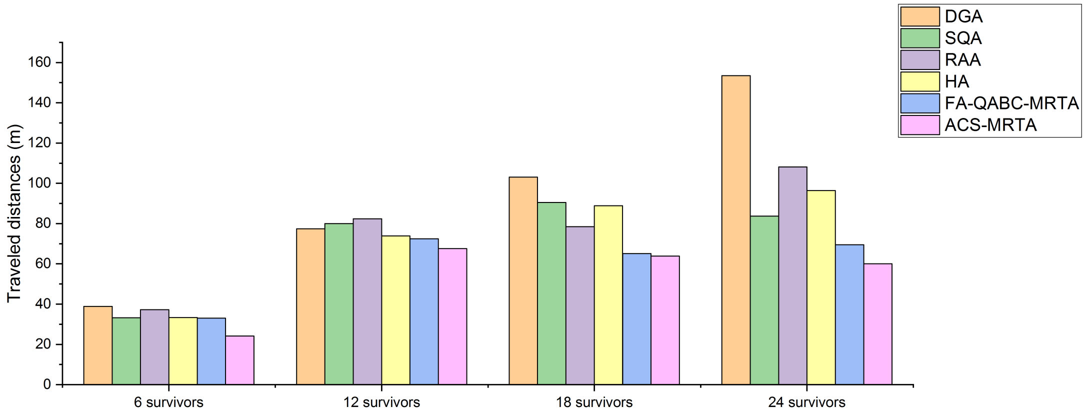

FIGURE 7. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of traveled distances (Case of 10 UAVs).

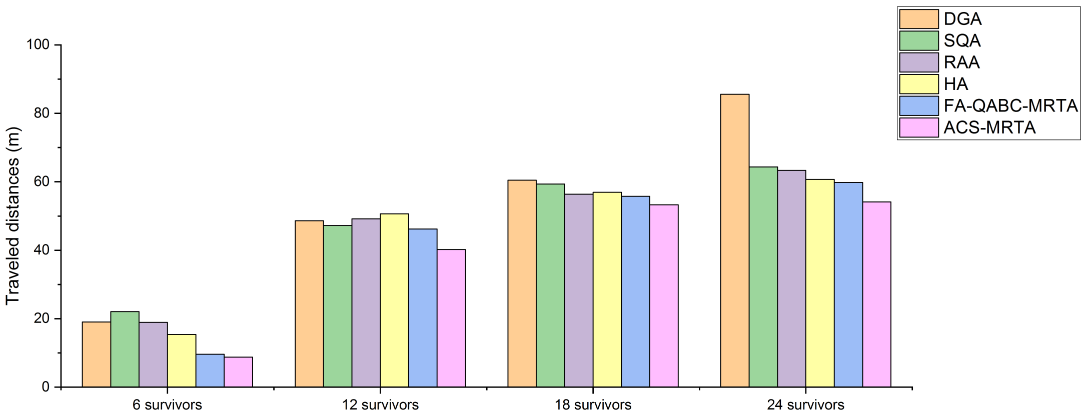

FIGURE 8. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of traveled distances (Case of 15 UAVs).

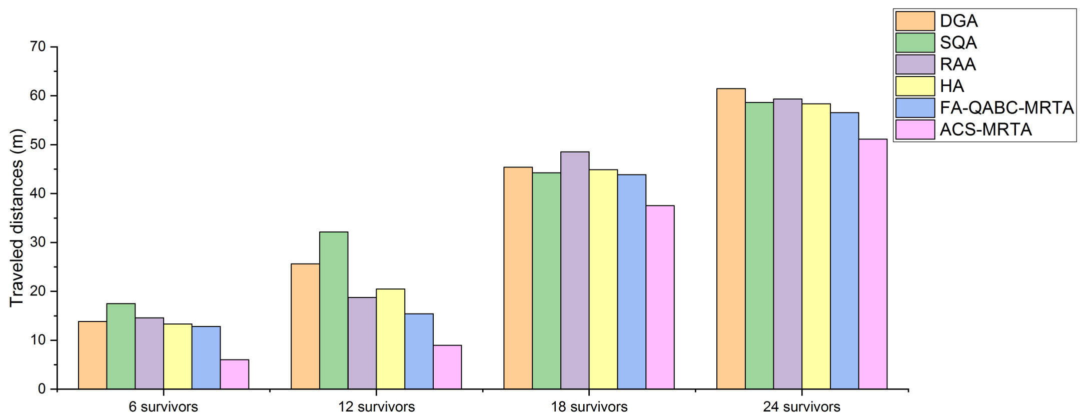

FIGURE 9. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of traveled distances (Case of 20 UAVs).

in terms of makespans, compared to HA, DGA, RAA and SQA. But, ACS-MRTA is clearly the best one for all simulation scenarios (even with large numbers of survivors and UAVs, makespans are approximately 200 seconds).

In multi-robot systems, the number of exchanged messages is another important metric for the evaluation of MRTA solutions: minimizing the number of exchanged messages means increasing the reliability. Figures 10, 11, 12 and 13

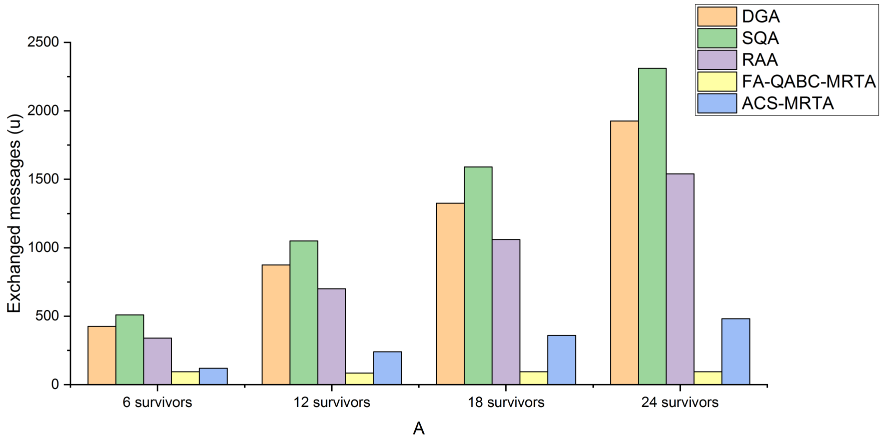

FIGURE 10. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of exchanged messages (Case of 5 UAVs).

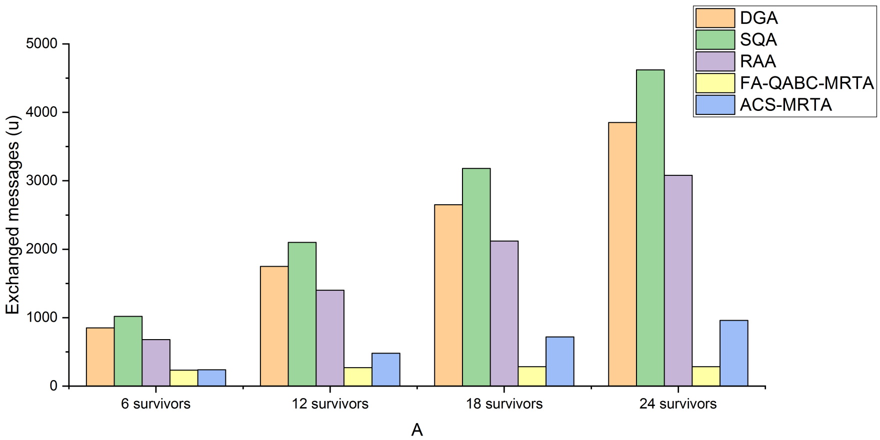

FIGURE 11. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of exchanged messages (Case of 10 UAVs).

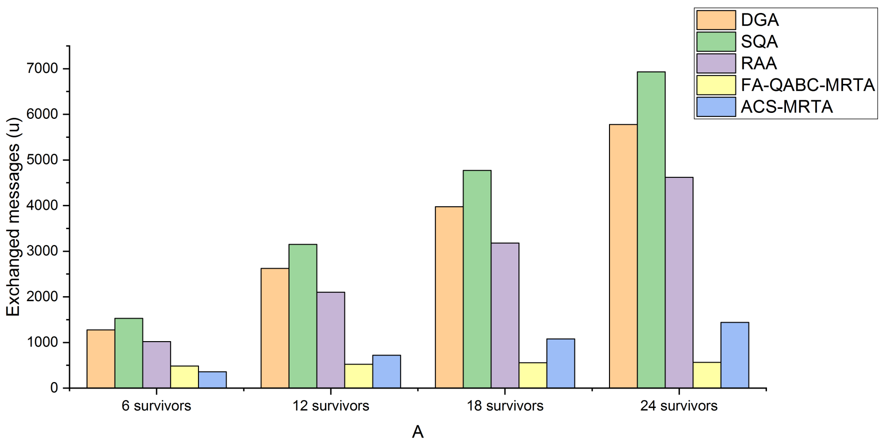

FIGURE 12. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of exchanged messages (Case of 15 UAVs).

show a comparison of solutions "DGA", "RAA", "SQA", "FA-QABC-MRTA" and "ACS-MRTA", in terms of exchanged messages. "HA" solution is not considered, because it is centralized: i.e. each UAV computes its own solution without interacting with other UAVs. The number of UAVs is set to 5, 10, 15 or 20. The number of survivors is

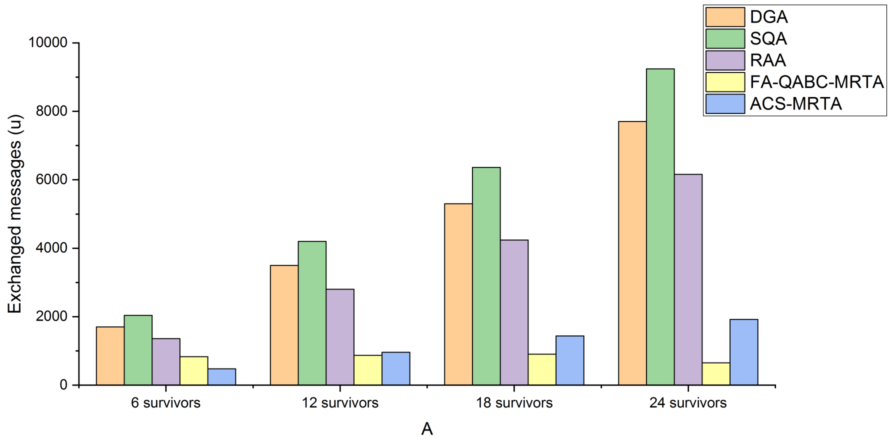

FIGURE 13. Comparisons of solutions HA, DGA, RAA, SQA, FA-QABC-MRTA and ACS-MRTA, in terms of exchanged messages (Case of 20 UAVs).

TABLE 3. The run-times of used algorithms.

Algorithm	Time complexity
DGA	$\mathcal{O}(n^2)$
SQA	$\mathcal{O}(n^4)$
RAA	$\mathcal{O}(n^2)$
HA	$\mathcal{O}(n^4)$
FA-QABC-MRTA	$\mathcal{O}(n^2)$
ACS-MRTA	$\mathcal{O}(n^3)$

set to 6, 12, 18 or 24. First, we notice that the solution "SQA" has the highest number of exchanged messages, followed by "DGA", "RAA", "ACS-MRTA", and finally "FA-QABC-MRTA". "FA-QABC-MRTA" is better than "ACS-MRTA", in terms of exchanged messages. The main reason of this situation is that the solution "FA-QABC-MRTA" uses a central units and UAVs are only allowed to communicate with it; however, the solution "ACS-MRTA" is fully distributed and each UAV is allowed to communicate with any UAV. Finally, it is important to mention that the numbers of exchanged messages produced by "ACS-MRTA" are very close to the ones exchanged by "FA-QABC-MRTA".

Table 3 represents time complexities of our proposed solution and algorithms used for comparisons, where $n = \max\{|S|, |U|\}$. We notice that the time complexity of all algorithms is polynomial. The time complexity of DGA, RAA and FA-QABC-MRTA is $\mathcal{O}(n^2)$. The time complexity of ACS-MRTA is $\mathcal{O}(n^3)$. The time complexity of HA and SQA is $\mathcal{O}(n^4)$. Time complexity of our solution is in the middle of the other algorithms. However, it outperforms all of them, in terms of makespans and travelled distances; and almost all of them, in terms of exchanged messages.

V. CONCLUSION AND PERSPECTIVES

We presented a distributed approach solution to the multi-robot task allocation problem, that combines the consensus-based bundle algorithm [10] and Ant Colony System [19]. We chose a case study where we have a number of survivors, which need to be rescued by some Unmanned Aerial Vehicles; that is to say: (a) survivors are the tasks; (b) UAVs are the robots; and (c) UAVs should rescue as many survivors as possible while optimizing makespans and travelled distances. Since the approach is based on the consensus-based bundle algorithm, thus it has two main consecutive phases. The first is performed once and the second is dynamically executed to maintain the best quality of generated solutions if changes occur in the environment. In the inclusion phase, each UAV selects survivors to rescue using the Ant Colony System [19]; therefore a bundle of survivors is built for each one. In the consensus phase, a coordination mechanism was proposed to resolve the conflicts generated in the consensus phase. UAVs exchange messages, share information and the Ant Colony System [19] is periodically executed if survivors are removed. The approach is implemented using Java programming language and JADE multi-agent Framework. Comparison studies among the proposed approach and five other algorithms [35], [36], [40], [58], [81], in terms of: (i) makespans (a gain of $77,5\%$); (ii) traveled distances (a gain of 40%); and (iii) exchanged messages (a gain of 75%), were conducted on 16 different cases to demonstrate the performance of the proposed solution.

As perspectives, we plan to further improve the proposed approach by using energetic, temporal, and spatial constraints on survivors and UAVs. Also, we aim to implement it on some real-life scenarios: using real UAVs and survivors.

REFERENCES

[1] N. Atay and B. Bayazit, "Mixed-integer linear programming solution to multi-robot task allocation problem," Tech. Rep. WUCSE-2006-54, 2006. [Online]. Available: https://openscholarship.wustl.edu/cse_research/205

[2] M. Badreldin, A. Hussein, and A. Khamis, "A comparative study between optimization and market-based approaches to multi-robot task allocation," *Adv. Artif. Intell.*, vol. 2013, pp. 1–11, 2013.

[3] S. Botelho and R. Alami, "M+: A scheme for multi-robot cooperation through negotiated task allocation and achievement," in *Proc. IEEE Int. Conf. Robot. Autom.*, vol. 2, Jan. 2003, pp. 1234–1239.

[4] T. Campbell, L. Johnson, and J. P. How, "Multiagent allocation of Markov decision process tasks," in *Proc. Amer. Control Conf.*, Jun. 2013, pp. 2356–2361.

[5] G. D. Celik and E. Modiano, "Dynamic vehicle routing for data gathering in wireless networks," in *Proc. 49th IEEE Conf. Decis. Control (CDC)*, Dec. 2010, pp. 2372–2377.

[6] L. Chaimowicz, M. Campos, and V. Kumar, "Dynamic role assignment for cooperative robots," in *Proc. IEEE Int. Conf. Robot. Autom.*, vol. 1, Jun. 2003, pp. 293–298.

[7] A. C. Chapman, R. A. Micillo, R. Kota, and N. R. Jennings, "Decentralised dynamic task allocation: A practical game: Theoretic approach," in *Proc. 8th Int. Conf. Auton. Agents Multiagent Syst.*, vol. 2, 2009, pp. 915–922.

[8] W.-H. Chen and C.-S. Lin, "A hybrid heuristic to solve a task allocation problem," *Comput. Oper. Res.*, vol. 27, no. 3, pp. 287–303, Mar. 2000.

[9] X. Chen, P. Zhang, G. Du, and F. Li, "A distributed method for dynamic multi-robot task allocation problems with critical time constraints," *Robot. Auto. Syst.*, vol. 118, pp. 31–46, Aug. 2019.

[10] H.-L. Choi, L. Brunet, and J. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Trans. Robot.*, vol. 25, no. 4, pp. 912–926, Aug. 2009.

[11] B. Coltin and M. Veloso, "Mobile robot task allocation in hybrid wireless sensor networks," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2010, pp. 2932–2937.

[12] P. Dasgupta, "Multi-robot task allocation for performing cooperative foraging tasks in an initially unknown environment," in *Innovations in Defence Support Systems-2: Socio-Technical Systems*, L. C. Jain, E. V. Aidman, and C. Abeynayake, Eds. Berlin, Germany: Springer, 2011, pp. 5–20, doi: 10.1007/978-3-642-17764-4_2.

[13] M. B. Dias, *Traderbots: A New Paradigm for Robust and Efficient Multirobot Coordination in Dynamic Environments*. Pittsburgh, PA, USA: Carnegie Mellon Univ., Jan. 2004, p. 153.

[14] M. B. Dias and A. Stentz, "A free market architecture for distributed control of a multirobot system," in *Proc. 6th Int. Conf. Intell. Auton. Syst. (IAS-6)*, 2000, pp. 115–122.

[15] M. Dias and A. Stentz, "Opportunistic optimization for market-based multirobot control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, vol. 3, Jun. 2003, pp. 2714–2720.

[16] Y. Ding, M. Zhu, Y. He, and J. Jiang, "An autonomous task allocation method of the multi-robot system," in *Proc. 9th Int. Conf. Control, Autom., Robot. Vis.*, 2006, pp. 1–6.

[17] U. Diwekar, "*Introduction to Applied Optimization*, vol. 22. New York, NY, USA: Springer, 2008. [Online]. Available: https://books.google.ae/books?id=nrdHC-7RPdEC

[18] M. Dorigo and T. Stutzle, *Ant Colony Optimization Theory. A Brandford Book*. London, U.K.: MIT Press, 2004.

[19] M. Dorigo and L. Gambardella, "Ant colony system: A cooperative learning approach to the traveling salesman problem," *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 53–66, Apr. 1997.

[20] J. Du, L. Zhou, P. Qu, Z. Shi, and L. Yang, "Task allocation in multi-agent systems with swarm intelligence of social insects," in *Proc. 6th Int. Conf. Natural Comput.*, vol. 8, Aug. 2010, pp. 4322–4326.

[21] S. S. Fatima and M. Wooldridge, "Adaptive task resources allocation in multi-agent systems," in *Proc. 5th Int. Conf. Auto. Agents (AGENTS)*, 2001, pp. 537–544.

[22] R. Faure, B. Lemaire, and C. Picouleau, *Précis De Recherche Opérationnelle*. Paris, France: Dunod, 2014.

[23] B. Gerkey and M. Mataric, "Sold!: Auction methods for multirobot coordination," *IEEE Trans. Robot. Automat.*, vol. 18, no. 5, pp. 758–768, Oct. 2002.

[24] B. P. Gerkey and M. J. Mataric, "A formal analysis and taxonomy of task allocation in multi-robot systems," *Int. J. Robot. Res.*, vol. 23, no. 9, pp. 939–954, Sep. 2004.

[25] A. Hussein, M. Adel, M. Bakr, O. M. Shehata, and A. Khamis, "Multi-robot task allocation for search and rescue missions," *J. Phys., Conf. Ser.*, vol. 570, no. 5, Dec. 2014, Art. no. 052006.

[26] E. G. Jones, M. Bernardine Dias, and A. Stentz, "Learning-enhanced market-based task allocation for oversubscribed domains," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2007, pp. 2308–2313.

[27] E. G. Jones, M. B. Dias, and A. Stentz, "Time-extended multi-robot coordination for domains with intra-path constraints," *Auton. Robot.*, vol. 30, no. 1, pp. 41–56, Jan. 2011.

[28] E. Jones, B. Browning, M. Dias, B. Argall, M. Veloso, and A. Stentz, "Dynamically formed heterogeneous robot teams performing tightly-coordinated tasks," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Jul. 2006, pp. 570–575.

[29] D. Juedes, F. Drews, L. Welch, and D. Fleeman, "Heuristic resource allocation algorithms for maximizing allowable workload in dynamic, distributed real-time systems," in *Proc. 18th Int. Parallel Distrib. Process. Symp.*, Jun. 2004, p. 117.

[30] N. Kalra and A. Martinoli, "Comparative study of market-based and threshold-based task allocation," in *Distributed Autonomous Robotic Systems 7*, M. Gini and R. Voyles, Eds. Tokyo, Japan: Springer, 2006, pp. 91–101.

[31] A. Khamis, A. Hussein, and A. Elmogy, "Multi-robot task allocation: A review of the state-of-the-art," in *Cooperative Robots and Sensor Networks 2015*, A. Koubâa and J. R. M. Dios, Eds. Cham, Switzerland: Springer, 2015, pp. 31–51, doi: 10.1007/978-3-319-18299-5_2.

[32] W. Kmiecik, M. Wojcikowski, L. Koszalka, and A. Kasprzak, "Task allocation in mesh connected processors with local search meta-heuristic algorithms," in *Intelligent Information and Database Systems*. Berlin, Germany: Springer, 2010, pp. 215–224, doi: 10.1007/978-3-642-12101-2_23.

[33] Y. Kong, M. Zhang, and D. Ye, "A group task allocation strategy in open and dynamic grid environments," in *Recent Advances in Agent-based Complex Automated Negotiation*, N. Fukuta, T. Ito, M. Zhang, K. Fujita, and V. Robu, Eds. Cham, Switzerland: Springer, 2016, pp. 121–139, doi: 10.1007/978-3-319-30307-9_8.

[34] G. A. Korsah, A. Stentz, and M. B. Dias, "A comprehensive taxonomy for multi-robot task allocation," *Int. J. Robot. Res.*, vol. 32, no. 12, pp. 1495–1512, Oct. 2013.

[35] H. W. Kuhn, "The hungarian method for the assignment problem," *Naval Res. Logistics Quart.*, vol. 2, nos. 1–2, pp. 83–97, 1955.

[36] W. Lenagh, P. Dasgupta, and A. Munoz-Melendez, "A spatial queuing-based algorithm for multi-robot task allocation," *Robotics*, vol. 4, no. 3, pp. 316–340, Sep. 2015.

[37] S. Lim and D. Rus, "Stochastic motion planning with path constraints and application to optimal agent, resource, and route planning," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2012, pp. 4814–4821.

[38] D. Liu and A. Kulatunga, "Simultaneous planning and scheduling for multi-autonomous vehicles," in *Evolutionary Scheduling*. Berlin, Germany: Springer, 2007, pp. 437–464.

[39] S.-H. Liu, Y. Zhang, H.-Y. Wu, and J. Liu, "Multi-robot task allocation based on swarm intelligence," *J. Jilin Univ. (Eng. Technol. Ed.)*, vol. 40, pp. 123-129, Jan. 2010.

[40] L. Luo, N. Chakraborty, and K. Sycara, "Competitive analysis of repeated greedy auction algorithm for online multi-robot task assignment," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2012, pp. 4792–4799.

[41] J. G. Manathara, P. B. Sujit, and R. W. Beard, "Multiple UAV coalitions for a search and prosecute mission," *J. Intell. Robot. Syst.*, vol. 62, no. 1, pp. 125–158, Apr. 2011.

[42] Z. Marszalek, "Performance test on triple heap sort algorithm," *Tech. Sci.*, vol. 1, no. 20, pp. 49–61, Jun. 2019.

[43] T. Michalak, J. Sroka, T. Rahwan, M. Wooldridge, P. McBurney, and N. R. Jennings, "A distributed algorithm for anytime coalition structure generation," in *Proc. 9th Int. Conf. Auto. Agents Multiagent Syst.*, vol. 1, 2010, pp. 1007–1014.

[44] A. R. Mosteo, "Multi-robot task allocation for service robotics: From unlimited to Ltd. communication range," Ph.D. dissertation, Universidad de Zaragoza, Zaragoza, Spain, 2010.

[45] A. R. Mosteo and L. Montano, "Simulated annealing for multi-robot hierarchical task allocation with flexible constraints and objective functions," in *Proc. Workshop Netw. Robot Syst., Toward Intell. Robot. Syst. Integr. Environ. (IROS)*, 2006, pp. 1–9.

[46] N. Nedjah, R. M. D. Mendonça, and L. D. M. Mourelle, "PSO-based distributed algorithm for dynamic task allocation in a robotic swarm," *Procedia Comput. Sci.*, vol. 51, pp. 326–335, 2015.

[47] E. Nunes, M. Manner, H. Mitiche, and M. Gini, "A taxonomy for task allocation problems with temporal and ordering constraints," *Robot. Auto. Syst.*, vol. 90, pp. 55–70, Apr. 2017.

[48] J. Parker, E. Nunes, J. Godoy, and M. Gini, "Exploiting spatial locality and heterogeneity of agents for search and rescue teamwork," *J. Field Robot.*, vol. 33, no. 7, pp. 877–900, 2016.

[49] L. Parker, "ALLIANCE: An architecture for fault tolerant multirobot cooperation," *IEEE Trans. Robot. Automat.*, vol. 14, no. 2, pp. 220–240, Apr. 1998.

[50] A. Petcu and B. Faltings, "Superstabilizing, fault-containing distributed combinatorial optimization," in *Proc. Nat. Conf. Artif. Intell.*, vol. 20. Menlo Park, CA, MA; London; AAAI Press, 2005, p. 449.

[51] S. D. Ramchurn, M. Polukarov, A. Farinelli, C. Truong, and N. R. Jennings, ''Coalition formation with spatial and temporal constraints,'' in *Proc. 9th Int. Conf. Auto. Agents Multiagent Syst.*, vol. 3, 2010, pp. 1181–1188.
[52] S. Sariel and T. Balch, ''A distributed multi-robot cooperation framework for real time task achievement,'' in *Distributed Autonomous Robotic Systems 7*, M. Gini and R. Voyles, Eds. Springer, 2006, pp. 187–196, doi: 10.1007/4-431-35881-1_19.
[53] P. Scerri, A. Farinelli, S. Okamoto, and M. Tambe, ''Allocating tasks in extreme teams,'' in *Proc. 4th Int. Joint Conf. Auto. Agents Multiagent Syst. (AAMAS)*, 2005, pp. 727–734.
[54] K. Tian Seow, N. Hai Dang, and D.-H. Lee, ''A collaborative multiagent taxi-dispatch system,'' *IEEE Trans. Autom. Sci. Eng.*, vol. 7, no. 3, pp. 607–616, Jul. 2010.
[55] Y. D. Sergeyev, R. G. Strongin, and D. Lera, *Introduction to Global Optimization Exploiting Space-Filling Curves*. New York, NY, USA: Springer, 2013.
[56] P. Shea, K. Alexander, and J. Peterson, ''Group tracking using genetic algorithms,'' in *Proc. 6th Int. Conf. Inf. Fusion*, 2003, pp. 680–687.
[57] O. Shehory and S. Kraus, ''Methods for task allocation via agent coalition formation,'' *Artif. Intell.*, vol. 101, nos. 1–2, pp. 165–200, May 1998.
[58] R. G. Smith, ''The contract net protocol: High-level communication and control in a distributed problem solver,'' *IEEE Trans. Comput.*, vol. C-29, no. 12, pp. 1104–1113, Dec. 1980.
[59] J. C. Spall, ''*Introduction to Stochastic Search and Optimization: Estimation, Simulation, and Control*, vol. 65. Hoboken, NJ, USA: Wiley, 2005.
[60] F. Tang and L. Parker, ''ASyMTRe: Automated synthesis of multi-robot task solutions through software reconfiguration,'' in *Proc. IEEE Int. Conf. Robot. Autom.*, Jan. 2006, pp. 1501–1508.
[61] F. Tang and L. E. Parker, ''A complete methodology for generating multi-robot task solutions using ASyMTRe-D and market-based task allocation,'' in *Proc. IEEE Int. Conf. Robot. Autom.*, Apr. 2007, pp. 3351–3358.
[62] J. Turner, Q. Meng, G. Schaefer, A. Whitbrook, and A. Soltoggio, ''Distributed task rescheduling with time constraints for the optimization of total task allocations in a multirobot system,'' *IEEE Trans. Cybern.*, vol. 48, no. 9, pp. 2583–2597, Sep. 2018.
[63] J. Wang, Y. Gu, and X. Li, ''Multi-robot task allocation based on ant colony algorithm,'' *J. Comput.*, vol. 7, no. 9, pp. 2160–2167, 2012.
[64] B. B. Werger and M. J. Mataric, ''Broadcast of local eligibility: Behavior-based control for strongly cooperative robot teams,'' in *Proc. 4th Int. Conf. Auto. Agents (AGENTS)*, vol. 3, 2000, pp. 21–22.
[65] A. Whitbrook, Q. Meng, and P. W. H. Chung, ''Reliable, distributed scheduling and rescheduling for time-critical, multiagent systems,'' *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 2, pp. 732–747, Apr. 2018.
[66] D. Yang and Z.-O. Wang, ''Improved ant algorithm for assignment problem,'' *J. Tianjin Univ.*, vol. 4, p. 20, Apr. 2004.
[67] X.-S. Yang, *Nature-Inspired Optimization Algorithms*. Amsterdam, The Netherlands: Elsevier, 2014.
[68] D. Yingying, H. Yan, and J. Jingping, ''Multi-robot cooperation method based on the ant algorithm,'' in *Proc. IEEE Swarm Intell. Symp. (SIS)*, Mar. 2004, pp. 14–18.
[69] Z. Yu, L. Shuhua, F. Shuai, and W. Di, ''A quantum-inspired ant colony optimization for robot coalition formation,'' in *Proc. Chin. Control Decis. Conf.*, Jun. 2009, pp. 626–631.
[70] D. Zhang, G. Xie, J. Yu, and L. Wang, ''Adaptive task assignment for multiple mobile robots via swarm intelligence approach,'' *Robot. Auto. Syst.*, vol. 55, no. 7, pp. 572–588, Jul. 2007.
[71] Y. Zhang and L. E. Parker, ''IQ-ASyMTRe: Forming executable coalitions for tightly coupled multirobot tasks,'' *IEEE Trans. Robot.*, vol. 29, no. 2, pp. 400–416, Apr. 2013.
[72] Y. Zhang and L. E. Parker, ''Multi-robot task scheduling,'' in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2013, pp. 2992–2998.
[73] Y. Zhang, L. E. Parker, and S. Kambhampati, ''Coalition coordination for tightly coupled multirobot tasks with sensor constraints,'' in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2014, pp. 1090–1097.
[74] W. Zhao, Q. Meng, and P. W. H. Chung, ''A heuristic distributed task allocation method for multivehicle multitask problems and its application to search and rescue scenario,'' *IEEE Trans. Cybern.*, vol. 46, no. 4, pp. 902–915, Apr. 2016.
[75] X. Zheng and S. Koenig, ''Reaction functions for task allocation to cooperative agents,'' in *Proc. 7th Int. Joint Conf. Auto. Agents Multiagent Syst.*, vol. 2, 2008, pp. 559–566.
[76] F. Zitouni, S. Harous, and R. Maamri, ''A distributed solution to the multi-robot task allocation problem using ant colony optimization and bat algorithm,'' in *Proc. Int. Conf. Modelling, Simulation Intell. Comput.* Dubai, UAE: Springer, 2020.
[77] F. Zitouni, S. Harous, and R. Maamri, ''Towards a distributed solution to the multi-robot task allocation problem with energetic and spatiotemporal constraints,'' *Comput. Sci.*, vol. 21, no. 1, pp. 3–24, 2020.
[78] F. Zitouni and R. Maamri, ''An adaptive protocol for dynamic allocation of tasks in a multi-robot system,'' in *Proc. Int. Conf. Adv. Aspects Softw. Eng. (ICAASE)*, Oct. 2016, pp. 128–133.
[79] F. Zitouni and R. Maamri, ''Cooperative learning-agents for task allocation problem,'' in *Interactive Mobile Communication Technologies and Learning*. Springer, 2017, pp. 952–968.
[80] F. Zitouni and R. Maamri, ''FA-SETPOWER-MRTA: A solution for solving the multi-robot task allocation problem,'' in *Proc. Comput. Intell. Appl., 6th IFIP TC 5 Int. Conf.* Oran, Algeria: Springer, May 2018, pp. 317–328.
[81] F. Zitouni, R. Maamri, and S. Harous, ''FA-QABC-MRTA: A solution for solving the multi-robot task allocation problem,'' *Intell. Service Robot.*, vol. 12, no. 4, pp. 407–418, Oct. 2019.
[82] F. Zitouni, M. Ramdane, and Z. Abdelhafid, ''Multi-robot task allocation with energetic, spatial and temporal constraints,'' *Int. J. Artif. Intell.*, vol. 17, no. 1, pp. 102–138, 2019.
[83] R. Zlot and A. Stentz, ''Market-based multirobot coordination for complex tasks,'' *Int. J. Robot. Res.*, vol. 25, no. 1, pp. 73–101, Jan. 2006.
[84] R. Zlot, A. Stentz, M. Dias, and S. Thayer, ''Multi-robot exploration controlled by a market economy,'' in *Proc. IEEE Int. Conf. Robot. Autom.*, vol. 3, Jun. 2003, pp. 3016–3023.
[85] R. M. Zlot and A. Stentz, ''An auction-based approach to complex task allocation for multirobot teams,'' Ph.D. dissertation, Carnegie Mellon Univ., Robot. Inst., Pittsburgh, PA, USA, 2006.

FAROUQ ZITOUNI obtained his Magister diploma in 2015 from Abdelhamid Mehri University, Constantine, Algeria. He is currently a Ph.D. student in Computer Science at Abdelhamid Mehri University, Constantine, Algeria. He is currently a Professor at the Department of Computer Science, Kasdi Merbah University – Ouargla, Algeria. He is working on the field of multi-robot task allocation and multi-robot systems. His main research interests include optimization and metaheuristics. He published some papers in international conferences and journals. Currently, he is visiting the UAE University.

SAAD HAROUS (Senior Member, IEEE) received the Ph.D. degree in computer science from Case Western Reserve University, Cleveland, OH, USA, in 1991. He has more than 30 years of experience in teaching and research in three different countries: USA, Oman, and UAE. He is currently a Professor with the College of Information Technology, United Arab Emirates University. His teaching interests include programming, data structures, design and analysis of algorithms, operating systems, and networks. He has published more than 160 journal and conference papers. His research interests include parallel and distributed computing, P2P delivery architectures, wireless networks, and the use of computers in education and processing Arabic language.

RAMDANE MAAMRI received the master's degree from the University of Minnesota, USA, and the Ph.D. degree in computer science from the University of Mentouri, Constantine, Algeria. He has participated in many national and international research projects and supervised a number of master's and Ph.D. students. He is currently an Associate Professor of computer science with the University of Constantine 2, Algeria. He is also the Head of the Software Technologies and Information Systems Department. His research interests include software engineering, verification and validation, web services, multiagent systems, and ambient systems.